

Genetic Algorithm for Reservoir Computing Optimization

Aida A. Ferreira, Teresa B. Ludermir

Abstract— This paper presents reservoir computing optimization using Genetic Algorithm. Reservoir Computing is a new paradigm for using artificial neural networks. Despite its promising performance, Reservoir Computing has still some drawbacks: the reservoir is created randomly; the reservoir needs to be large enough to be able to capture all the features of the data. We propose here a method to optimize the choice of global parameters using genetic algorithm. This method was applied on a real problem of time series forecasting. The time of search for the best global parameters with GA was just 22.22% of the time- consuming task to an exhausting search of the same parameters.

I. INTRODUCTION

Genetic Algorithms (GAs) are algorithms of global optimization based on the mechanisms of natural selection and on genetics. Although GAs perform a randomized strategy, they use a structured parallel search to look for the points of high aptitude, i.e., points at which the function to be minimized or maximized has values relatively high or low. In spite his randomized strategy, GAs explore historical information to find new search points where better performance are expected.

The proposal of this work is to use the technique of GA to search the global parameters that optimize the solution of real problems with Reservoir Computing (RC).

Reservoir Computing [1][2] is a new paradigm with promising results [3][4]. RC offers an intuitive methodology for using the temporal processing power of recurrent neural networks (RNN) without the inconvenience of training them. Originally introduced independently as Liquid State Machine (LSM) [1] or Echo State Network (ESN) [2], the basic concept is to construct a RNN at random and leave the weights unchanged. A separate linear regression function is trained on the response of the reservoir to the input signals using pseudo-inverse. The underlying idea is that a randomly constructed reservoir offers a complex nonlinear dynamic transformation of the input signals which allows the readout to extract the desired output using a simple linear mapping [3].

When we want to solve a task using reservoir computing,

Manuscript received December 15, 2008.

Aida A. Ferreira, is with the Federal Center of Technologic Education of Pernambuco, Av Professor Luis Freire, 500, Cidade Universitária, Cep:50.740-530 – Recife – PE – Brazil (corresponding author to provide phone: 55-81-21251781; e-mail: aidaferreira@cefetpe.br) and Center of Informatics (CIn), Federal University of Pernambuco (UFPE), P.O. Box 7851, Cidade Universitária, Cep: 50.740-530 – Recife – PE – Brazil (corresponding authors to provide phone: 55-81-21268430; e-mail: aaf@cin.ufpe.br)

Teresa B. Ludermir is with the Center of Informatics (CIn), Federal University of Pernambuco (UFPE), P.O. Box 7851, Cidade Universitária, Cep: 50.740-530 – Recife – PE – Brazil (corresponding authors to provide phone: 55-81-21268430; e-mail: tbl@cin.ufpe.br).

we have to choose not only the node quantity but also the node type, the interconnection topology and spectral radius. Several metrics have been described in the literatures that are supposed to offer a priori indication of how the reservoir will perform without explicitly having to apply it to a task. However, a clear indication of which metric values are optimal for which task is not yet available, and the values of these metrics have not yet been linked to the actual reservoir dynamics [5].

A theoretical property is defined for potentially interesting reservoirs called for echo state property [1], which expresses – informally stated – the fact that the influence of inputs on reservoir states fades away gradually. Further, an upper and lower bound are defined for the echo state property that is very easy to compute and depend only on the weight matrix of the reservoirs. However, this property alone is not enough to guarantee optimal performance for a given problem, and the search for a good reservoir requires experience and can take some time [5].

For all computation reported in this work we used Matlab. We used the Genetic Algorithm toolbox and the code available in [6] to create the RC networks.

The structure of this paper is as follows. In Section 2, we present a short review of Reservoir Computing. Then, Genetic Algorithm for Reservoir Computing Optimization is analyzed in Section 3. Database and variable arrangement are discussed in Section 4. In Section 5, we describe the methodology used. Next, in Section 6, results are shown. Finally, in Section 7, conclusions and future works are drawn.

II. RESERVOIR COMPUTING

Reservoir Computing offers an intuitive methodology for using the temporal processing power of RNN without the hassle of training them [3]. RNN are examples of neural computation models that handle time without the need for preprocessing delay lines. RNNs have recurrent connections between the processing elements (PEs) creating internally the memory required to store the history of the input patterns [7], [8]. They have been widely used in many applications such as system identification and control of dynamical systems [9], [10], [11]. In recent years, a number of approaches for processing of time-varying inputs have been proposed that utilize the complex dynamic inherent in some recurrent networks architecture. Among the most prominent examples of such architectures are Liquid State Machine (LSM) [1] and Echo State Network (ESN) [2]. Here we will refer to both RC.

In RC the reservoir is a fixed, randomly structured recurrent network that receives time-varying input on which certain computations are to be performed. The reservoir fulfills two functions. First, it nonlinearity transforms input

streams into high-dimensional activation patterns. Second, it exhibits a fading memory of recent inputs. These properties are exploited by a simple linear readout mechanism that can be trained to perform interesting computation on input time series. To this end, the linear readout is often trained with standard linear regression techniques [12].

Fig. 1 shows a diagram of an ESN with M input units, N internal PEs and L output units. The Value of the input unit at time n is $u(n)$, of internal units are $x(n)$, and output units are $y(n)$.

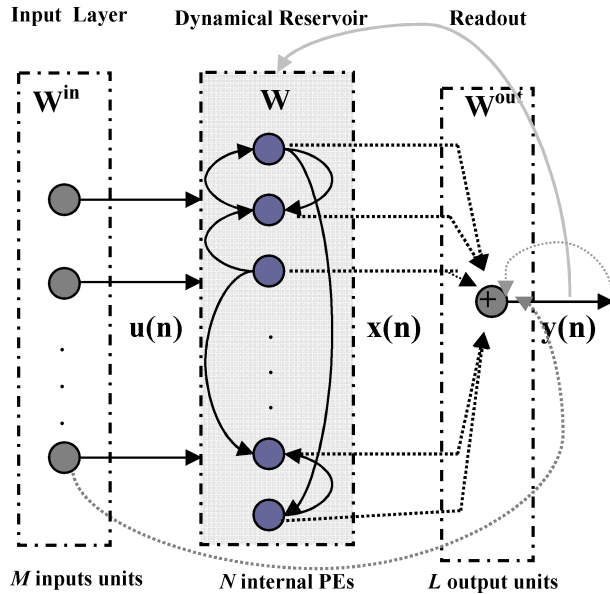


Fig. 1. The basic ESN architecture assumed in this work. Shaded arrows indicate optional connections. Dotted arrows indicate trainable connections.

III. GENETIC ALGORITHM FOR RESERVOIR COMPUTING OPTIMIZATION

Genetic algorithm (GA) is a popular method of optimization based on the theory of the natural evolution [13]. GA is (i) approximation (heuristic) algorithm, i.e., it does not guarantee the finding of an optimal; (ii) it is blind, i.e., it does not know when an optimal solution is found, and therefore, must be told when to stop; (iii) it occasionally accepts bad moves (movements away from the optimal solution); (iv) it can easily implement a diversity of problems, all that is required is to have a suitable solution representation, a cost function, and a mechanism to traverse the search space; and (v) under certain conditions, it can asymptotically converge to an optimal solution.

The search process of GA involves a sequence of iterations, where a set of solutions passes through the selection processes and reproduction. To create the next generation, the GA selects certain individuals in the current population, called parents, and uses them to create individuals in the next generation, called children. Typically, the algorithm is more likely to select parents that have better

fitness values.

The GA creates three types of children for the next generation: Elite children are the individuals in the current generation with the best fitness values. These individuals automatically survive to the next generation. Crossover children are created by combining the vectors of a pair of parents. Mutation children are created by introducing random changes, or mutations, to a single parent. Fig. 2 illustrates the three types of children.

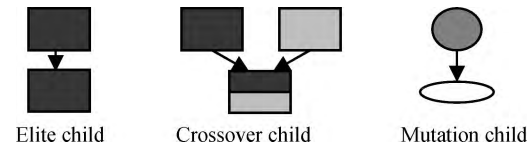


Fig. 2. Types of children in genetic algorithm.

The search heuristics of GA incorporates specific knowledge about the domain of the problem to accomplish the optimization process. It tolerates several non-deterministic elements that help the search to escape of local minima. It has an appropriate cost function for each problem that turns it very widely used.

As the GAs are very efficient for searching optimal solutions, or approximately optimal, in a wide variety of problems (they do not impose many of the limitations found in the traditional methods [14]) we decided to investigate the use of GA to optimize the choice of the RC global parameters to the problem of time series forecast. Usually, the search of those parameters is made in an exhausting way or through random experiments, which in general takes a long time to be accomplished and consumes a lot of computational resource.

The parameters choose to be optimized by GA are: amount of the nodes in the reservoir (N), type of node activation function in the analog node of the reservoir (tanh or sigmoid), spectral radius, existence or not of interconnection among the input layer and output layer and feedback connection in the output layer.

The size N of dynamical reservoir (DR) should reflect both the length T of training data, and the difficulty of the task. As a rule of thumb, N should not exceed an order of magnitude of $T/10$ to $T/2$ (the more regular-periodic the training data, the closer to $T/2$ can N be chosen). This is a simple precaution against overfitting. Furthermore, more difficult tasks require larger N [20].

The diligent choice of the spectral radius α of the dynamical reservoir (DR) weight matrix is of crucial importance for the eventual success of ESN training. This is because α is intimately connected to the intrinsic timescale of the dynamics of the DR state. Small α means that one has a fast DR, large α (i. e., close to unity) means that one has a slow DR. The intrinsic timescale of the task should match the DR timescale. Typically α needs to be hand-tuned by trying out several settings [20].

The topology is an important parameter that determines the behaviour and performance of an RC system. In the RC

Toolbox, the topology of the system is completely defined by the topology structure. This structure can be depicted graphically as a single large matrix, see in Fig. 3. As the figure shows, the toolbox allows connecting inputs, reservoir and outputs on the one hand to reservoir and outputs on the other hand. The user is free to fill in the components he wants to achieve different setups, with the exception of the reservoir to output connections [6].

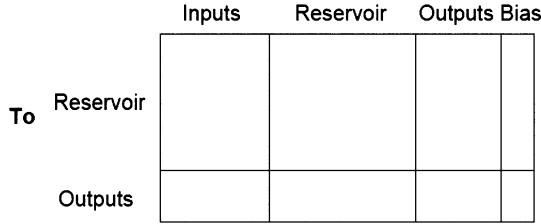


Fig. 3. Autocorrelation analysis of wind speed time series.

In the RC Toolbox, the user can choose between analog or spiking nodes. For analog nodes, the model always performs a weighted sum of the input values followed by nonlinearity. The nonlinearities available in the toolbox are: linear, sign, piecewise linear, tanh and fermi [6]. In this work we used only analog nodes and tanh or sign nonlinearities.

IV. DATABASE AND VARIABLE ARRANGEMENT

In this work real data from the project SONDA (System of National Organization of Ambient Data - <http://www.cptec.inpe.br/sonda/>) have been used to create the model. SONDA is a project of the National Institute of Space Research (INPE) on implementation of a physical infrastructure and human resources in order to gather and to improve the database of the resources of solar and wind energy in Brazil.

The wind power model in energy planning is based on statistical operation of the wind farms considering the wind regime. For the Northeast Region of Brazil it has been shown in [15] that the wind characteristics are very directional wind speed. In that way we chose a time series of wind speed to accomplish the experiments of optimization of the global parameters of RC. The series is constituted by the hourly wind speed obtained by the wind headquarters of Triunfo. This city in Brazil is located in the highest altitude of Pernambuco, 1,123 meters, and its series of wind speed presents the higher average speed of the state, 11.83 m/s. The series is constituted by the hourly wind speed in the period from January 01, 2006 to April 30, 2007, which amount to 11,640 patterns and it was used previously [16] to create forecast models with random experiments of the global parameters.

In forecast models that are univariate, the autocorrelation function for the series defines its applicability and acts mainly in the statistical models. Using this correlation function, it is possible to identify the dependence among series data, which facilitate the data analysis.

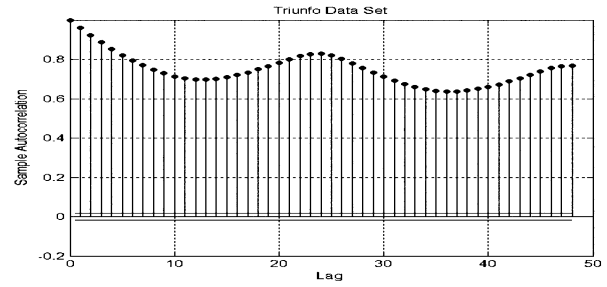


Fig. 4. Autocorrelation analysis of wind speed time series.

The database was analyzed by the autocorrelation function of the data of the wind series in order to define the forecast horizon. It can be easily verified through a qualitative analysis of the graph in Fig. 4 that the forecast models with small forecast horizon tend to have a superior performance. Besides, it can be seen that in the proximity of 24 hours the autocorrelation function of the series increases again. This behavior characterizes the seasonality of winds among the day periods.

Before creating the system, the base was preprocessed and the values of the average hourly speeds was transformed, in the same way that was done in [19], to fall in a limited range $[-1, 1]$. The values were transformed as in (1).

$$x_{trans} = \frac{(y_{max} - y_{min}) * (x - x_{min})}{(x_{max} - x_{min})} + y_{min} \quad (1)$$

where, x_{trans} is the value transformed in a limited range $[-1, 1]$; y_{max} is the maximum value of the interval, 1; y_{min} is the minimum value of the interval, 0; x_{max} and x_{min} are the maximum and minimum values of the series and; x is the original value. To the maximum value of the series was added 20%. Thus, the maximum value accepted by the model is equal to the maximum value found in the database increased by 20% and the minimum value is zero.

Twenty-four-step-forward predictor of the average hourly wind speed was chosen, thus the series presented a good correlation index and 24-step-forward is a good interval to the operation system planning.

V. METHODOLOGY

The proposed methodology aims for the optimization of the global parameters of RC. In this section, the methodology will be developed for optimization of five global parameters. We explained the parameters to be optimized in section III: amount of the nodes in the reservoir (N), type of node activation function in the analog node of the reservoir (tanh or sigmoid), spectral radius, existence or not of interconnection among the input layer and output layer and feedback connection in the output layer.

In the accomplished simulations, each solution s is codified by a vector that is formed by:

- N - number of node in the reservoir (50, 600)
- nf - node activation function in the reservoir (1- Tanh or 2- Sign)
- sr - spectral radius (0.75 to 0.95)
- io - interconnection among the input and output layers (0- yes, 1-no)
- oo - feedback connection in the output layer (0- yes, 1-no)

The initial solution is generated randomly with 100 individuals. A new solution s' is generated through genetic micro-evolutions with g_n generations from an initial solution $s = (N, nf, sr, io, oo)$.

The individuals are classified by the Rank Based Fitness Scaling, that is, the scores returned by the fitness function are converted to a value range of rank that is more useful for selection function [17]. The selection function uses the scaled fitness values to select the parents of the next generation. The selection function assigns a higher probability of selection to individuals with higher scaled values. The fitness function defined in our experiments was measured by the percentage of the mean-squared error (MSE) in the training set as shown in (1).

$$MSE_{\%} = 100 \times \frac{L_{max} - L_{min}}{N \cdot P} \sum_{p=1}^P \sum_{i=1}^N (L_{pi} - T_{pi})^2 \quad (2)$$

where L_{max} and L_{min} are the maximum and minimum of the hourly speed values in the data, respectively; N is the number of output units of the ANN; P is the total number of patterns in the training set; L_{pi} and T_{pi} are actual and desired (target) outputs of the i -th neuron in the output layer respectively.

The selection function chooses parents for the next generation based on their scaled values from the fitness scaling function. An individual can be selected more than once as a parent, in which case, it contributes its genes to more than one child. The selection option was *stochastic uniform*, in this case the parents are chosen by a line in which each parent corresponds to a section of the line of length proportional to its scaled value. The algorithm moves along the line in steps of equal size. At each step, the algorithm allocates a parent from the section it lands on. The first step is a uniform random number less than the step size.

The elitism was used with value equal to five. That means that five individuals with the best fitness values in the current generation are guaranteed to survive to the next generation. These individuals are called elite children.

In the experiments, the Population size is 100, the Elite count is 5, and the Crossover fraction is 0.8. The following outline summarizes the pseudo-code of the proposed algorithm for optimization of the global parameters with GA:

- I. The algorithm begins by creating a random initial population of 100 individuals $s = (N, nf, sr, io, oo)$. For each individual s one network is created with the parameters and the *Fitness* function of this network is assessed in the training data.
- II. The algorithm then creates a sequence of new populations. At each step, the algorithm uses the individuals in the current generation to create the next population. To create the new population, the algorithm performs the following steps:
 - a. Score each member of the current population by computing its fitness value.
 - b. Scale the raw fitness scores to convert them into a more usable range of values.
 - c. Select members, called parents, based on

their fitness. Some of the individuals in the current population that have high fitness are chosen as elite. These elite individuals are passed to the next population.

- d. Generate children from the parents. Children are generated either by making random changes to a single parent — mutation — or by combining the vector inputs of a pair of parents — crossover. Replace the current population with the children to form the next generation.
- e. The algorithm stops when one of the stopping criteria is met. The stopping criteria used were: the *maximum number of generations*, in this case 15 generations.

VI. RESULTS

The model was created by the Hybrid Intelligent System - HIS, whose network architecture was chosen by the GA, developed specially to accomplish this objective and the model for forecasting by RC. The GA attempts to optimize the global parameters of RC, acting directly in the choice of the number of nodes in reservoir, in the kind of activation function in reservoir nodes (tanh or sign), in the size of spectral radius, in the connection between input layer and output layer and in the feedback connection in output layer. **Fig. 5** shows a comparison of the time-consuming task to search the best parameters configuration using exhausting search and using GA. The time of the search with GA was 22.22% of the time-consuming task to an exhausting search of the same parameters. While we search around 720000 networks in exhaustive search, in GA search we only create 1600 networks.

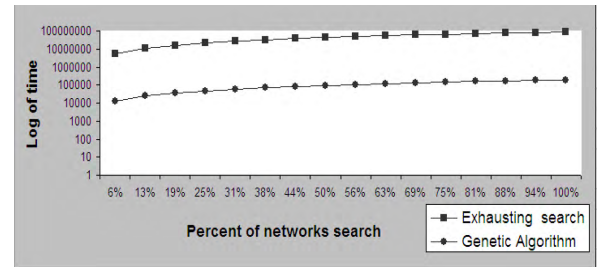


Fig. 5. Comparison of the time-consuming task to search the best parameters configuration using exhausting search and using GA.

Table 1 presents a comparison of the results obtained in [16], where random experiments were accomplished to choose the parameters, with the results obtained using the suitable parameters by the proposed methodology. As the data set was split in 10 chunks, each one was used separately as a testing set, while the rest was used to train the readout. The MSE value in training set showed that GA search was better than random experiments, although the MSE value in test set showed that the obtained results were similar.

TABLE I
COMPARISON OF PERFORMANCE OF EACH METHOD

Model	Training Set MSE (%)	Test Set MSE (%)
Parameters chose by random experiments in [16]	0.94	0.96
Parameters chose by Genetic Algorithm	0.91	0.95

The best network obtained in [16] is composed by 400 nodes in reservoir, scaled to a spectral radius of 0.9, analog nodes with nonlinearity function “tanh” and with connection between input layer and output layer and feedback connection in output layer. The best network obtained in GA search is composed by 547 nodes in reservoir, scaled to a spectral radius of 0.947, analog nodes with nonlinearity function “tanh” and with feedback connection in output layer. The great advantage of using GA for the search of the global parameters of RC is the reductions of the time and the computational effort to accomplish the search. We can improve these results by specification of a Fitness function more appropriate for RC and using new stop criteria for GA search method.

Fig. 6 presents the best and mean fitness values in each generation. Probably the number of generation could be lower if we had used different stop criteria for GA search method.

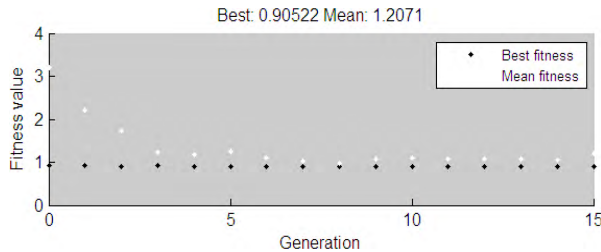


Fig. 6. The best and mean fitness values in each generation.

VII. CONCLUSIONS AND FUTURE WORKS

This work presented a first attempt of optimization of RC using GA. The use of GA showed to be a good option to search the global parameters of these kind of networks, because, although there are metric values and heuristics which are able to indicate the choice of some parameters, it still there is not a clear indication of which metric values are optimal for which task.

With the use of GA integrated to RC, we created an intelligent hybrid system that is capable to choose the best configuration of global parameters for the networks. The advantages of the hybrid system developed are that the choice does not depend on the analyst's experience, the time-consuming task for finding the parameters and the computational effort were reduced.

As the first optimization attempt combining RC with GA for wind speed forecasting, we identified several future works that can be developed, which are: specification of a

Fitness function more appropriate for RC; use of new stop criteria; use of this methodology in others data bases, increase the number of nonlinearities functions to be search.

REFERENCES

- [1] W. Maass, T. Natschlager, H. Markram, “Real-time computing without stable states: A new framework for neural computation based on perturbations”, in *Neural Computation*, vol. 14(11), pp. 2531–2560, 2002.
- [2] H. Jaeger, “The echo state approach to analyzing and training recurrent neural networks” in *Tech. Rep. GMD 148*, 2001, German National Resource Center for Information Technology.
- [3] B. Schrauwen, J. Defour, D. Verstraeten, J. Van Campenhout, “The introduction of time-scales in reservoir computing, applied to isolated digits recognition” in *LNCS*, vol. 4668, Part I, pp. 471–479, 2007.
- [4] E. A. Antonelo, B. Schrauwen, X. Dutoit, D. Stroobandt, M. Nuttin, “Event detection and location in mobile robot navigation using reservoir computing” in *Proc. International Conference on Artificial Neural Networks*, vol. 4668, Part II, pp. 660–669, 2007.
- [5] D. Verstraeten, B. Schrauwen, M. D’Haene, D. Stroobandt, “An experimental unification of reservoir computing methods” in *Neural Networks*, vol. 20(3), pp. 391–403, 2007.
- [6] B. Schrauwen, D. Verstraeten, M. D’Haene, “Reservoir Computing Toolbox Manual”. Available: <http://www.clis.ugent.be/rct>
- [7] H. Jaeger, H. Hass, “Harnessing nonlinearity: Predicting chaotic systems and Saving Energy in Wireless Communication” in *Science*, vol. 304(5667), pp. 78–80, 2004.
- [8] R. Sacchi, M. C. Ozturk, J. C. Principe, A. A. F. M. Carneiro, I. N. da Silva, “Water inflow forecasting using the echo state network: a Brazilian case study” in *Proc. International Joint Conference on Neural Networks*, Orlando, Florida, pp. 2403–2408, 2007.
- [9] G. Kechriotis, E. Zervas, E. S. Manolakas, “Using recurrent neural networks for adaptive communication channel equalization” in *IEEE Transactions on Neural Networks*, vol. 5 (2), pp. 267–278, 1994.
- [10] G. V. Puskorius, L. A. Feldkamp, “Neurocontrol of nonlinear dynamical systems with Kalman filter trained recurrent networks” in *IEEE Transactions on Neural Networks*, vol. 5(2), pp. 279–297, 1994.
- [11] A. Delgado, C. Kambhampati, K. Warwick, “Dynamic recurrent neural network for system identification and control” in *IEEE Proceedings: Control Theory and Applications*, vol. 142(4), pp. 307–314, 1995.
- [12] A. Lazar, G. Pipa, J. Triesch, “Fading Memory and time series prediction in recurrent networks with different forms of plasticity” in *Neural Networks*, vol. 20(3), pp. 312–322, 2007.
- [13] T. Bäck, “Evolutionary Algorithms in Theory and Practice” in *New York: Oxford University Press*, 1996.
- [14] J. Holland, “Adaptation in natural and artificial systems. Ann Arbor, MI” in *The University of Michigan Press*, 1992.
- [15] G. Rodrigues, “Wind Characteristics of the Northeast Region – Analysis, models and application to wind farm projects” in M.S.c Dep. Mechanical Eng., UFPE, Brazil, 2003. (in Portuguese)
- [16] A. A. Ferreira, T. B. Ludermir, “Using Reservoir Computing for Forecasting Time Series: Brazilian Case Study” in *Proc. 8th International Conference on Hybrid Intelligent Systems*, HIS 2008, vol. 1. pp. 602–607, 2008.
- [17] J. E. Baker, “Reducing bias and inefficiency in the selection algorithm” in *Proc. Second International Conference on Genetic Algorithms and Their Applications*, Lawrence Erlbaum Associates, pp. 14–21, 1987.
- [18] E. Vonk, L. C. Jain, R. P. Johnson, “Automatic Generation of Neural Network Architecture Using Evolutionary Computation” in *World Scientific*, Singapore, 1997.
- [19] A. A. Ferreira, T. B. Ludermir, R. R. B. De Aquino, M. M. S. Lira, O. N. Neto, “Investigating the use of reservoir computing for forecasting the hourly wind speed in short-term”, in *Proc. International Joint Conference on Neural Networks*, Hong Kong, pp. 1649–1656, 2008.
- [20] H. Jaeger, “Tutorial on training recurrent neural networks, covering BPPT, RTRL, EKF and the Echo State Network Approach”, *Tech. Rep. No. 159*, Bremen: German National Research Center for Information Technology, 2002. Available : <http://www.faculty.iubremen.de/hjaeger/pubs/ESNTutorial.pdf>